

# 40. 同定後 fullsim レポート生成

solar\_ws0129-main

2026-07-29

## Contents

|                                                |           |
|------------------------------------------------|-----------|
| <b>1 対象</b>                                    | <b>3</b>  |
| <b>2 このファイルを読む理由</b>                           | <b>3</b>  |
| <b>3 呼び出し関係</b>                                | <b>3</b>  |
| 3.1 どこから呼ばれるか                                  | 3         |
| 3.2 次に読むと理解が進むファイル                             | 3         |
| 3.3 同一リポジトリ内の主要依存                              | 3         |
| <b>4 ソース冒頭の把握</b>                              | <b>3</b>  |
| 4.1 代表コード断片                                    | 3         |
| <b>5 主要構造の読み取り</b>                             | <b>4</b>  |
| 5.1 主要クラス・関数                                   | 4         |
| 5.2 ファイルを上から読んだときの定義順                          | 5         |
| 5.3 import 群の詳細                                | 5         |
| <b>6 このファイルを読む前に必要な基礎知識</b>                    | <b>7</b>  |
| 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する                 | 7         |
| 6.2 名前、代入、型、参照                                 | 7         |
| 6.3 関数、引数、戻り値、スコープ                             | 7         |
| 6.4 module、package、import、console_scripts      | 8         |
| 6.5 例外、try/finally、with、資源解放                   | 8         |
| 6.6 list、dict、deque、NumPy 配列、pandas DataFrame  | 8         |
| 6.7 CLI、PowerShell、Bash、環境変数、終了コード             | 9         |
| 6.8 Cross-Entropy Method を式と実装で理解する            | 9         |
| 6.9 SoC、内部抵抗、端子電圧、温度、MHE                       | 9         |
| 6.10 freshness、filter、guard、fallback、fail-safe | 10        |
| 6.11 CSV/YAML の data contract と検証              | 10        |
| 6.12 単体試験、SILS、replay、観測可能性                    | 10        |
| 6.13 制御、状態、入力、モデル予測制御 MPC                      | 10        |
| <b>7 関数・クラスを上から順に解説</b>                        | <b>11</b> |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 7.1      | L38 関数 <code>resolve_path</code>                         | 11        |
| 7.2      | L50 関数 <code>latex_escape</code>                         | 12        |
| 7.3      | L71 関数 <code>rel_display</code>                          | 13        |
| 7.4      | L77 関数 <code>locate_package_dir</code>                   | 14        |
| 7.5      | L88 関数 <code>locate_fit_summary</code>                   | 15        |
| 7.6      | L119 関数 <code>rms</code>                                 | 16        |
| 7.7      | L127 関数 <code>day_block_bootstrap_rmse</code>            | 17        |
| 7.8      | L175 関数 <code>load_replay_diagnostics</code>             | 19        |
| 7.9      | L238 関数 <code>locate_fullsim_manifest</code>             | 21        |
| 7.10     | L266 関数 <code>resolve_manifest_artifact</code>           | 23        |
| 7.11     | L277 関数 <code>load_fullsim_summary</code>                | 24        |
| 7.12     | L401 関数 <code>audit_fullsim_detail</code>                | 26        |
| 7.13     | L477 関数 <code>interpolate_at_distance</code>             | 29        |
| 7.14     | L496 関数 <code>moving_speed_in_interval</code>            | 30        |
| 7.15     | L511 関数 <code>build_human_mpc_distance_comparison</code> | 31        |
| 7.16     | L549 関数 <code>build_daily_progress_comparison</code>     | 32        |
| 7.17     | L570 関数 <code>write_human_mpc_distance_plot</code>       | 34        |
| 7.18     | L627 関数 <code>md_table</code>                            | 36        |
| 7.19     | L645 関数 <code>tex_kv_table</code>                        | 37        |
| 7.20     | L661 関数 <code>tex_df_table</code>                        | 38        |
| 7.21     | L698 関数 <code>build_report</code>                        | 40        |
| 7.22     | L1560 関数 <code>main</code>                               | 44        |
| 7.23     | CLI 引数                                                   | 46        |
| <b>8</b> | <b>処理の流れ</b>                                             | <b>46</b> |
| <b>9</b> | <b>このファイルの位置づけ</b>                                       | <b>46</b> |

## 1 対象

| 項目          | 内容                                                               |
|-------------|------------------------------------------------------------------|
| ファイル        | scripts/generate_fit_fullsim_report.py                           |
| ソース SHA-256 | 0b02f300fe27dd5806e0686c58564ef85c115b501f13b58fc898d882f60c54fc |
| 種別          | Python                                                           |
| 区分          | report                                                           |
| 役割          | identification run と replay/fullsim の結果をまとめ、説明用レポートへ整形する。        |
| 起動文脈        | fit の結果説明と評価集約に使う。                                               |

## 2 このファイルを読む理由

identification run と replay/fullsim の結果をまとめ、説明用レポートへ整形する。

- 同定結果と full simulation を一つの説明資料へまとめる。

## 3 呼び出し関係

### 3.1 どこから呼ばれるか

- 手動レポート生成
- 後処理パイプライン

### 3.2 次に読むと理解が進むファイル

- 特記事項なし。

### 3.3 同一リポジトリ内の主要依存

- scripts/audit\_identification\_residuals.py
- scripts/build\_bwsc2025\_fitted\_package.py

## 4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

### 4.1 代表コード断片

```
from scripts.audit_identification_residuals import weather_and_cruise_metrics

def resolve_path(base_dir: Path, raw: str) -> Path:
    path = Path(str(raw or "").strip())
    if not path:
        return base_dir
    if path.is_absolute():
        return path
```

```

rooted = (ROOT / path).resolve()
if rooted.exists():
    return rooted
return (base_dir / path).resolve()

def latex_escape(text: object) -> str:
    value = str(text)
    repl = {
        "\\": r"\textbackslash{}",
        "&": r"\&",
        "%": r"\%",
        "$": r"\$",
    }

```

## 5 主要構造の読み取り

主要関数は `resolve_path`, `latex_escape`, `rel_display`, `locate_package_dir`, `locate_fit_summary`, `rms`, `day_block_bootstrap_rmse`, `load_replay_diagnostics`。CLI 引数宣言は 4 件。

### 5.1 主要クラス・関数

| 行   | 種別       | 名前                                               | 補足                                                                                                                                            |
|-----|----------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 38  | function | <code>resolve_path</code>                        | args: <code>base_dir</code> , <code>raw</code>                                                                                                |
| 50  | function | <code>latex_escape</code>                        | args: <code>text</code>                                                                                                                       |
| 71  | function | <code>rel_display</code>                         | args: <code>path</code> , <code>base_dir</code>                                                                                               |
| 77  | function | <code>locate_package_dir</code>                  | args: <code>profile_yaml</code>                                                                                                               |
| 88  | function | <code>locate_fit_summary</code>                  | args: <code>package_dir</code> , <code>profile_yaml</code>                                                                                    |
| 119 | function | <code>rms</code>                                 | args: <code>values</code>                                                                                                                     |
| 127 | function | <code>day_block_bootstrap_rmse</code>            | args: <code>df</code> , <code>residual_column</code>                                                                                          |
| 175 | function | <code>load_replay_diagnostics</code>             | args: <code>package_dir</code> , <code>replay_csv</code>                                                                                      |
| 238 | function | <code>locate_fullsim_manifest</code>             | args: <code>package_dir</code> , <code>profile_yaml</code>                                                                                    |
| 266 | function | <code>resolve_manifest_artifact</code>           | args: <code>manifest_path</code> , <code>raw</code> , <code>package_dir</code>                                                                |
| 277 | function | <code>load_fullsim_summary</code>                | args: <code>package_dir</code> , <code>profile_yaml</code> , <code>manifest_path</code>                                                       |
| 401 | function | <code>audit_fullsim_detail</code>                | args: <code>package_dir</code> , <code>manifest_path</code> , <code>manifest</code>                                                           |
| 477 | function | <code>interpolate_at_distance</code>             | args: <code>df</code> , <code>column</code> , <code>distance_km</code>                                                                        |
| 496 | function | <code>moving_speed_in_interval</code>            | args: <code>df</code> , <code>column</code> , <code>start_km</code> , <code>end_km</code>                                                     |
| 511 | function | <code>build_human_mpc_distance_comparison</code> | args: <code>replay_df</code> , <code>fullsim_df</code> , <code>retire_distance_km</code>                                                      |
| 549 | function | <code>build_daily_progress_comparison</code>     | args: <code>replay_df</code> , <code>fullsim_df</code>                                                                                        |
| 570 | function | <code>write_human_mpc_distance_plot</code>       | args: <code>replay_df</code> , <code>fullsim_df</code> , <code>comparison</code> , <code>output_path</code> , <code>retire_distance_km</code> |
| 627 | function | <code>md_table</code>                            | args: <code>df</code>                                                                                                                         |

|      |          |              |                                 |
|------|----------|--------------|---------------------------------|
| 645  | function | tex_kv_table | args: items                     |
| 661  | function | tex_df_table | args: df, title                 |
| 698  | function | build_report | args: package_dir, profile_yaml |
| 1560 | function | main         |                                 |

## 5.2 ファイルを上から読んだときの定義順

| 行    | 内容                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 18   | matplotlib.use(...) を実行する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 21   | ROOT に Path(__file__).resolve().parents[1] の結果を代入する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 22   | 条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 38   | 関数 resolve_path を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 50   | 関数 latex_escape を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 71   | 関数 rel_display を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 77   | 関数 locate_package_dir を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 88   | 関数 locate_fit_summary を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 119  | 関数 rms を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 127  | 関数 day_block_bootstrap_rmse を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 175  | 関数 load_replay_diagnostics を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 238  | 関数 locate_fullsim_manifest を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 266  | 関数 resolve_manifest_artifact を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 277  | 関数 load_fullsim_summary を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 356  | DETAIL_REQUIRED_COLUMNS に {'lower_command_index', 'time_utc', 'step_dt_sec', 'detail_target_dt_sec', 'outer_step_requested_dt_sec', 'outer_step_actual_dt_sec', 'outer_step_boundary_reason', 's_kmh', 'upper_speed_cmd_kmh', 'lower_speed_cmd_kmh', 'v_exec_kmh', 'soc', 'G_poa', 'Tamb_C', 'Tcell_C', 'P_pv', 'P_aux', 'P_vehicle_load_w', 'P_pack', 'I', 'V', 'OCV', 'Rint', 'Rline', 'eff_drv', 'eff_reg', 'F_aero', 'F_roll', 'F_grade', 'P_inertia', 'param_m', 'param_CdA', 'param_Crr', 'param_P_aux', 'param_E_nom_Wh', 'map_drive_eff_map', 'map_regen_eff_map', 'map_rint_map', 'map_panel_eff_map', 'map_mppt_eff_map', 'map_ocv_soc_map'} の結果を代入する。 |
| 401  | 関数 audit_fullsim_detail を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 477  | 関数 interpolate_at_distance を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 496  | 関数 moving_speed_in_interval を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 511  | 関数 build_human_mpc_distance_comparison を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 549  | 関数 build_daily_progress_comparison を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 570  | 関数 write_human_mpc_distance_plot を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 627  | 関数 md_table を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 645  | 関数 tex_kv_table を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 661  | 関数 tex_df_table を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 698  | 関数 build_report を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 1560 | 関数 main を定義する。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 1604 | 条件 __name__ == '__main__' を判定し、真なら内部処理を行う。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## 5.3 import 群の詳細

| 行 | import 文 | このファイルで読む理由 |
|---|----------|-------------|
|---|----------|-------------|

|    |                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2  | <code>from__future__<br/>_importannotations</code>                                                                                                                                                                                                                     | 型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。                                                                                                                                                                                                                                                                             |
| 4  | <code>importargparse</code>                                                                                                                                                                                                                                            | CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L1561。                                                                                                                                                                                                                                                                                                    |
| 5  | <code>importjson</code>                                                                                                                                                                                                                                                | manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L290, L1592。                                                                                                                                                                                                                                                                                  |
| 6  | <code>importmath</code>                                                                                                                                                                                                                                                | Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での使用位置は少ないか、間接利用である。                                                                                                                                                                                                                                                                            |
| 7  | <code>importos</code>                                                                                                                                                                                                                                                  | パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L74。                                                                                                                                                                                                                                                                                                              |
| 8  | <code>importsys</code>                                                                                                                                                                                                                                                 | sys モジュールを利用するため。このファイル内での主な使用位置は L22, L23。                                                                                                                                                                                                                                                                                                              |
| 9  | <code>importtextwrap</code>                                                                                                                                                                                                                                            | textwrap モジュールを利用するため。このファイル内での主な使用位置は L647, L675, L688, L1555。                                                                                                                                                                                                                                                                                          |
| 10 | <code>frompathlibimportPath</code>                                                                                                                                                                                                                                     | ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L21, L38, L39, L71, L77, L88, L175, L238, ...。                                                                                                                                                                                                                                                                      |
| 11 | <code>fromtypingimportDict,<br/>Iterable</code>                                                                                                                                                                                                                        | typing から Dict, Iterable を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L133, L175, L281, L326, L405, L645。                                                                                                                                                                                                                                                    |
| 13 | <code>importmatplotlib</code>                                                                                                                                                                                                                                          | matplotlib モジュールを利用するため。このファイル内での主な使用位置は L18。                                                                                                                                                                                                                                                                                                            |
| 14 | <code>importnumpyasnp</code>                                                                                                                                                                                                                                           | 数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L121, L124, L163, L165, L167, L168, L169, L216, ...。                                                                                                                                                                                                                                                             |
| 15 | <code>importpandasaspd</code>                                                                                                                                                                                                                                          | CSV や時刻列の読み込み・整形・再サンプリングに使うため。このファイル内での主な使用位置は L119, L120, L128, L142, L145, L180, L181, L182, ...。                                                                                                                                                                                                                                                      |
| 16 | <code>importyaml</code>                                                                                                                                                                                                                                                | profile、stop、schedule、summary YAML を読み書きするため。このファイル内での主な使用位置は L90, L239, L707, L708, L750, L847, L958, L988。                                                                                                                                                                                                                                             |
| 19 | <code>importmatplotlib.<br/>pyplotasplt</code>                                                                                                                                                                                                                         | matplotlib.pyplot モジュールを利用するため。このファイル内での主な使用位置は L579, L623。                                                                                                                                                                                                                                                                                              |
| 25 | <code>fromscripts.build_<br/>bwsc2025_fitted_<br/>packageimportBATTERY_E_<br/>NOM_MAX_WH, BATTERY_E_NOM_<br/>MIN_WH, BATTERY_ETA_CHARGE_<br/>MAX, BATTERY_ETA_CHARGE_<br/>MIN, BATTERY_RINT_SCALE_<br/>MAX, BATTERY_RINT_SCALE_MIN,<br/>compile_tex, ensure_dir</code> | build_bwsc2025_fitted_package.py から BATTERY_E_NOM_MAX_WH, BATTERY_E_NOM_MIN_WH, BATTERY_ETA_CHARGE_MAX, BATTERY_ETA_CHARGE_MIN, BATTERY_RINT_SCALE_MAX, BATTERY_RINT_SCALE_MIN, compile_tex, ensure_dir を読み込み、このファイルの内部処理を分担させるため。実体ファイルは scripts/build_bwsc2025_fitted_package.py。このファイル内での主な使用位置は L756, L796, L798, L800, L806, L807, L808, L809, ...。 |

|    |                                                                                                        |                                                                                                                                                                                          |
|----|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 35 | <code>fromscripts.audit_<br/>identification_<br/>residualsimportweather_<br/>and_cruise_metrics</code> | audit_identification_residuals.py か ら<br>weather_and_cruise_metrics を読み込み、このフ<br>ァイルの内部処理を分担させるため。実体ファイル<br>は scripts/audit_identification_residuals.py。このファ<br>イル内での主な使用位置は L720, L721。 |
|----|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

## 6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

### 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()  
alias = node  
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

#### 根拠資料

- [Python 公式チュートリアル: Classes](#)

### 6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

#### 根拠資料

- [Python 公式チュートリアル: Classes](#)

### 6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘\*args’、‘\*\*kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが `'self.z'` を更新すればオブジェクトの状態が残るが、単に `'z'` を更新しただけならその呼出しのローカル値である。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.4 module、package、import、console\_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。`'from .model import SolarCarModel'` の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。`'def'` や `'class'` は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の `'console_scripts'` は、端末で使う実行可能名と `'package.module:function'` を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼び
```

## 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

## 6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。`'try/except'` は想定した異常を処理し、`'finally'` は成功・失敗にかかわらず後始末を行う。

`'with open(...) as f:'` は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

`'except Exception: pass'` はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

## 6.6 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 `'(N)'`、候補集団なら `'(population, N)'` となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

## 6.7 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar\_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS\_DOMAIN\_ID、RMW\_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

### 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

## 6.8 Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった elite 候補から分布を更新する反復的な確率最適化である。このリポジトリの upper\_solver.py は各制御点速度を独立正規分布で生成し、上下限へ clip する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$
$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left( \text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで  $\mathcal{E}_g$  は cost が小さい上位  $K$  候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 hybrid\_bounded\_minimize は、deterministic seed を評価し、上位候補を L-BFGS-B で局所 refine し、設定と seed 間不一致に応じて CEM を実行し、最後に再度局所 refine する。したがって CEM 単独ではなく hybrid solver である。

CEM で落とした候補を永久保存しないこと自体は通常の最適化として自然だが、off-nominal 状態からの再利用には別の policy library 設計が必要である。状態を無制限に全組合せ保存する代わりに、SoC、進捗、時刻、予報誤差、停止状態などの scenario を設計し、近傍 policy を検索して MPC で再最適化する。

### 根拠資料

- [Rubinstein and Kroese: The Cross-Entropy Method](#)
- [SciPy 公式: scipy.optimize.minimize](#)

## 6.9 SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{\text{oc}}(z, T) - IR_{\text{total}}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T)I\Delta t}{Q_{\text{eff}}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

## 根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

### 6.10 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

### 6.11 CSV/YAML の data contract と検証

data contract は列名、型、単位、timezone、欠損可否、並び順、重複、許容範囲、先頭行、encoding を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、map、route、forecast、stop、schedule はそれぞれ grain が異なる。生成時に schema validation、物理範囲、時間単調性、route 範囲、coverage を検査し、検査結果を artifact として残す。

学習用データと独立検証データを分離し、RMSE だけでなく bias、時系列残差、energy 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

### 6.12 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

## 根拠資料

- ROS 2 Humble 公式: [rqt\\_graph](#)
- ROS 2 Humble 公式: [Recording and playing back data](#)
- ROS 2 公式: [Executors](#)

### 6.13 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を  $x$ 、操作入力を  $u$ 、外乱・予報を  $w$  とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から  $N$  ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPCを使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

## 根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

## 7 関数・クラスを上から順に解説

### 7.1 L38 関数 `resolve_path`

- 行範囲: L38–L47
- 所属: module 直下
- 定義: `resolve_path(base_dir:Path, raw:str)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `exists`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `(base_dir / path).resolve() / base_dir / path / rooted`
- 主なローカル代入名: `path`, `rooted`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `path` に `Path(str(raw or "").strip())` の結果を代入する。
2. 条件 `not path` を判定し、真なら内部処理を行う。
3. `base_dir` を返す。
4. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
5. `path` を返す。
6. `rooted` に `(ROOT / path).resolve()` の結果を代入する。
7. 条件 `rooted.exists()` を判定し、真なら内部処理を行う。
8. `rooted` を返す。
9. `(base_dir / path).resolve()` を返す。

#### 代表コード断片

```
def resolve_path(base_dir: Path, raw: str) -> Path:
    path = Path(str(raw or "").strip())
    if not path:
        return base_dir
    if path.is_absolute():
        return path
    rooted = (ROOT / path).resolve()
    if rooted.exists():
        return rooted
    return (base_dir / path).resolve()
```

## 7.2 L50 関数 latex\_escape

- 行範囲: L50–L68
- 所属: module 直下
- 定義: latex\_escape(text:object)->str
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: items, replace, str
- 戻り値の要点: value
- 主なローカル代入名: dst, repl, src, value
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. value に str(text) の結果を代入する。
2. repl に {'\\': '\\textbackslash{ }', '&': '\\&', '%': '\\%', '\$': '\\\$', '#': '\\#', '\_': '\\\_', '{': '\\{', '}': '\\}', '~': '\\textasciitilde{ }', '^': '\\textasciicircum{ }'} の結果を代入する。
3. repl.items() を順に走査し、各要素を (src, dst) に入れて処理する。
4. value に value.replace(src, dst) の結果を代入する。
5. value に value.replace('/', '\\allowbreak{ }) の結果を代入する。
6. value に value.replace('\\\_', '\\\_\\allowbreak{ }) の結果を代入する。
7. value を返す。

#### 代表コード断片

```
def latex_escape(text: object) -> str:
    value = str(text)
    repl = {
        "\\": r"\textbackslash{}",
        "&": r"\&",
        "%": r"\%",
        "$": r"\$",
        "#": r"\#",
        "_": r"\_",
        "{": r"\{",
        "}": r"\}",
        "~": r"\textasciitilde{}",
        "^": r"\textasciicircum{}",
    }
    for src, dst in repl.items():
        value = value.replace(src, dst)
    value = value.replace("/", r"/\allowbreak{}")
    value = value.replace(r"\_", r"\_\allowbreak{}")
    return value
```

### 7.3 L71 関数 rel\_display

- 行範囲: L71-L74
- 所属: module 直下
- 定義: `rel_display(path:Path|None, base_dir:Path)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `relpath`, `replace`
- 戻り値の要点: `os.path.relpath(path, base_dir).replace('\', '/') / 'not found'`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

#### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

#### 上から順の処理

1. 条件 `path is None` を判定し、真なら内部処理を行う。
2. ‘not found’ を返す。
3. `os.path.relpath(path, base_dir).replace('\', '/')` を返す。

#### 代表コード断片

```
def rel_display(path: Path | None, base_dir: Path) -> str:
    if path is None:
        return "not found"
    return os.path.relpath(path, base_dir).replace("\\", "/")
```

### 7.4 L77 関数 `locate_package_dir`

- 行範囲: L77-L85
- 所属: module 直下
- 定義: `locate_package_dir(profile_yaml: Path) -> Path`
- docstring: Return the owning project package even for versioned run profiles.
- このブロックが直接呼ぶ主な関数・メソッド: `is_dir`, `resolve`
- 戻り値の要点: `profile_yaml.parent / candidate`
- 主なローカル代入名: `candidate`, `profile_yaml`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

#### この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

#### 上から順の処理

1. `profile_yaml` に `profile_yaml.resolve()` の結果を代入する。
2. `(profile_yaml.parent, *profile_yaml.parents)` を順に走査し、各要素を `candidate` に入れて処理する。
3. 条件 `candidate.parent.name != 'project_packages'` を判定し、真なら内部処理を行う。
4. `Continue` 文を実行する。
5. 条件 `(candidate / 'data').is_dir()` and `(candidate / 'outputs').is_dir()` を判定し、真なら内部処理を行う。
6. `candidate` を返す。
7. `profile_yaml.parent` を返す。

#### 代表コード断片

```
def locate_package_dir(profile_yaml: Path) -> Path:
    """Return the owning project package even for versioned run profiles."""
    profile_yaml = profile_yaml.resolve()
    for candidate in (profile_yaml.parent, *profile_yaml.parents):
        if candidate.parent.name != "project_packages":
            continue
        if (candidate / "data").is_dir() and (candidate / "outputs").is_dir():
            return candidate
    return profile_yaml.parent
```

## 7.5 L88 関数 `locate_fit_summary`

- 行範囲: L88–L116
- 所属: module 直下
- 定義: `locate_fit_summary(package_dir:Path,profile_yaml:Path|None=None)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `FileNotFoundError`, `exists`, `get`, `glob`, `read_text`, `safe_load`, `sorted`, `stat`, `str`, `strip`
- 戻り値の要点: `candidates[0] / preferred / tagged`
- 主なローカル代入名: `candidates`, `output_tag`, `preferred`, `profile`, `tagged`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError('generic fit summary not found')`, `FileNotFoundError(f'fit summary for identification.output_tag={output_tag!r} not found: {tagged}')`
- 制御構造の規模: 条件分岐 5、ループ 0、try 0

### この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

### 上から順の処理

1. 条件 `profile_yaml is not None and profile_yaml.exists()` を判定し、真なら内部処理を行う。
2. `profile` に `yaml.safe_load(profile_yaml.read_text(encoding='utf-8')) or {}` の結果を代入する。
3. `output_tag` に `str((profile.get('identification', {}) or {}).get('output_tag', '') or '').strip()` の結果を代入する。
4. 条件 `output_tag` を判定し、真なら内部処理を行う。
5. `tagged` に `package_dir / 'outputs' / 'identification' / 'runs' / output_tag / f'{package_dir.name}_generic_fit_summary.yaml'` の結果を代入する。
6. 条件 `tagged.exists()` を判定し、真なら内部処理を行う。
7. `tagged` を返す。
8. `FileNotFoundError(f'fit summary for identification.output_tag={output_tag!r} not found: {tagged}')` を送出する。
9. `preferred` に `package_dir / 'outputs' / 'identification' / f'{package_dir.name}_generic_fit_summary.yaml'` の結果を代入する。
10. 条件 `preferred.exists()` を判定し、真なら内部処理を行う。
11. `preferred` を返す。
12. `candidates` に `sorted((package_dir / 'outputs' / 'identification').glob('*_generic_fit_summary.yaml'), key=lambda p: p.stat().st_mtime, reverse=True)` の結果を代入する。
13. 条件 `not candidates` を判定し、真なら内部処理を行う。
14. `FileNotFoundError('generic fit summary not found')` を送出する。
15. `candidates[0]` を返す。

#### 代表コード断片

```
def locate_fit_summary(package_dir: Path, profile_yaml: Path | None = None) -> Path:
    if profile_yaml is not None and profile_yaml.exists():
        profile = yaml.safe_load(profile_yaml.read_text(encoding="utf-8")) or {}
        output_tag = str((profile.get("identification", {}) or {}).get("output_tag", "") or
        "").strip()
        if output_tag:
            tagged = (
                package_dir
                / "outputs"
                / "identification"
                / "runs"
                / output_tag
                / f"{package_dir.name}_generic_fit_summary.yaml"
            )
            if tagged.exists():
                return tagged
            raise FileNotFoundError(
                f"fit summary for identification.output_tag={output_tag!r} not found: {
                tagged}"
            )
        preferred = package_dir / "outputs" / "identification" / f"{package_dir.name}_generic_fit_summary.yaml"
        if preferred.exists():
            return preferred
        candidates = sorted(
            (package_dir / "outputs" / "identification").glob("*_generic_fit_summary.yaml"),
            key=lambda p: p.stat().st_mtime,
            reverse=True,
        )
        if not candidates:
            raise FileNotFoundError("generic fit summary not found")
        return candidates[0]
```

## 7.6 L119 関数 rms

- 行範囲: L119–L124
- 所属: module 直下
- 定義: `rms(values:pd.Series)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`, `mean`, `sqrt`, `to_numeric`, `to_numpy`
- 戻り値の要点: `float(np.sqrt(np.mean(arr ** 2))) / float('nan')`
- 主なローカル代入名: `arr`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

### この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. arr に `pd.to_numeric(values, errors='coerce').to_numpy(dtype=float)` の結果を代入する。
2. arr に `arr[np.isfinite(arr)]` の結果を代入する。
3. 条件 `arr.size == 0` を判定し、真なら内部処理を行う。
4. `float('nan')` を返す。
5. `float(np.sqrt(np.mean(arr ** 2)))` を返す。

### 代表コード断片

```
def rms(values: pd.Series) -> float:
    arr = pd.to_numeric(values, errors="coerce").to_numpy(dtype=float)
    arr = arr[np.isfinite(arr)]
    if arr.size == 0:
        return float("nan")
    return float(np.sqrt(np.mean(arr ** 2)))
```

## 7.7 L127 関数 `day_block_bootstrap_rmse`

- 行範囲: L127-L172
- 所属: module 直下
- 定義: `day_block_bootstrap_rmse(df:pd.DataFrame,residual_column:str,*,draws:int=10000,seed:int=20260714)->Dict[str,float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `agg`, `assign`, `astype`, `default_rng`, `dropna`, `float`, `groupby`, `int`, `integers`, `len`, `quantile`
- 戻り値の要点: `{'rmse': float(np.sqrt(sse.sum() / count.sum()))`, `'ci95_min': float(np.quantile(sampled_rmse, 0.025))`, `'ci95_max': float(np.quantile(sampled_rmse, 0.975))`, `'day_blocks': int(len(blocks))`, `'draws': int(draws)}` / `{'rmse': float('nan')`, `'ci95_min': float('nan')`, `'ci95_max': float('nan')`, `'day_blocks': 0`, `'draws': 0}` / `{'rmse': float('nan')`, `'ci95_min': float('nan')`, `'ci95_max': float('nan')`, `'day_blocks': 0`, `'draws': 0}`
- 主なローカル代入名: `blocks`, `count`, `rng`, `sampled_rmse`, `samples`, `sse`, `work`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

### この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

## 上から順の処理

1. 条件 `df.empty or 'local_date' not in df.columns or residual_column not in df.columns` を判定し、真なら内部処理を行う。
2. `{'rmse': float('nan'), 'ci95_min': float('nan'), 'ci95_max': float('nan'), 'day_blocks': 0, 'draws': 0}` を返す。
3. `work` に `pd.DataFrame({'local_date': df['local_date'].astype(str), 'residual': pd.to_numeric(df[residual_column], errors='coerce')}).dropna()` の結果を代入する。
4. 条件 `work.empty` を判定し、真なら内部処理を行う。
5. `{'rmse': float('nan'), 'ci95_min': float('nan'), 'ci95_max': float('nan'), 'day_blocks': 0, 'draws': 0}` を返す。
6. `blocks` に `work.assign(sq=lambda frame: frame['residual'] ** 2).groupby('local_date').agg(sse=('sq', 'sum'), count=('sq', 'size'))` の結果を代入する。
7. `sse` に `blocks['sse'].to_numpy(dtype=float)` の結果を代入する。
8. `count` に `blocks['count'].to_numpy(dtype=float)` の結果を代入する。
9. `rng` に `np.random.default_rng(seed)` の結果を代入する。
10. `samples` に `rng.integers(0, len(blocks), size=(int(draws), len(blocks)))` の結果を代入する。
11. `sampled_rmse` に `np.sqrt(sse[samples].sum(axis=1) / count[samples].sum(axis=1))` の結果を代入する。
12. `{'rmse': float(np.sqrt(sse.sum() / count.sum())), 'ci95_min': float(np.quantile(sampled_rmse, 0.025)), 'ci95_max': float(np.quantile(sampled_rmse, 0.975)), 'day_blocks': int(len(blocks)), 'draws': int(draws)}` を返す。

## 代表コード断片

```
def day_block_bootstrap_rmse(
    df: pd.DataFrame,
    residual_column: str,
    *,
    draws: int = 10_000,
    seed: int = 20260714,
) -> Dict[str, float]:
    if df.empty or "local_date" not in df.columns or residual_column not in df.columns:
        return {
            "rmse": float("nan"),
            "ci95_min": float("nan"),
            "ci95_max": float("nan"),
            "day_blocks": 0,
            "draws": 0,
        }
    work = pd.DataFrame(
        {
            "local_date": df["local_date"].astype(str),
            "residual": pd.to_numeric(df[residual_column], errors="coerce"),
        }
    ).dropna()
    if work.empty:
        return {
            "rmse": float("nan"),
            "ci95_min": float("nan"),
            "ci95_max": float("nan"),
            "day_blocks": 0,
            "draws": 0,
        }
    blocks = (
        work.assign(sq=lambda frame: frame["residual"] ** 2)
```

```

        .groupby("local_date")
        .agg(sse=("sq", "sum"), count=("sq", "size"))
    )
    sse = blocks["sse"].to_numpy(dtype=float)
...

```

## 7.8 L175 関数 load\_replay\_diagnostics

- 行範囲: L175–L235
- 所属: module 直下
- 定義: `load_replay_diagnostics(package_dir:Path, replay_csv:Path|None=None) -> Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `abs`, `agg`, `astype`, `copy`, `exists`, `fillna`, `float`, `getattr`, `groupby`, `mean`, `nlargest`
- 戻り値の要点: `{'replay_csv': replay_csv, 'frame': df, 'clean_frame': clean, 'daily': daily, 'worst_power': worst_power, 'worst_voltage': worst_voltage}` / `{'replay_csv': replay_csv, 'frame': pd.DataFrame(), 'clean_frame': pd.DataFrame(), 'daily': pd.DataFrame(), 'worst_power': pd.DataFrame(), 'worst_voltage': pd.DataFrame()}`
- 主なローカル代入名: `clean`, `daily`, `df`, `fallback_ts`, `replay_csv`, `ts`, `worst_power`, `worst_voltage`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 0、`try` 0

### この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

### 上から順の処理

1. `replay_csv` に `replay_csv` or `package_dir / 'outputs' / 'identification' / 'replay_validation.csv'` の結果を代入する。
2. 条件 `not replay_csv.exists()` を判定し、真なら内部処理を行う。
3. `{'replay_csv': replay_csv, 'frame': pd.DataFrame(), 'clean_frame': pd.DataFrame(), 'daily': pd.DataFrame(), 'worst_power': pd.DataFrame(), 'worst_voltage': pd.DataFrame()}` を返す。
4. `df` に `pd.read_csv(replay_csv, low_memory=False)` の結果を代入する。
5. `fallback_ts` に `pd.to_datetime(df['time_utc'], format='mixed', utc=True, errors='coerce').dt.tz_convert('Australia/Darwin')` の結果を代入する。
6. 条件 `'time_local' in df.columns` を判定し、真なら内部処理を行う。
7. `ts` に `pd.to_datetime(df['time_local'], format='mixed', errors='coerce')` の結果を代入する。

8. 条件 `getattr(ts.dt, 'tz', None) is not None` を判定し、真なら内部処理を行う。
9. `ts` に `ts.dt.tz_localize(None)` の結果を代入する。
10. `ts` に `ts.fillna(fallback_ts)` の結果を代入する。
11. 上の条件が偽の場合:
12. `ts` に `fallback_ts` の結果を代入する。
13. `df['local_date']` に `ts.dt.strftime('%Y-%m-%d')` の結果を代入する。
14. `df['power_resid_w']` に `pd.to_numeric(df['battery_power_w_obs'], errors='coerce') - pd.to_numeric(df['battery_power_w_pred'], errors='coerce')` の結果を代入する。
15. `df['voltage_resid_v']` に `pd.to_numeric(df['battery_voltage_v_obs'], errors='coerce') - pd.to_numeric(df['battery_voltage_v_pred'], errors='coerce')` の結果を代入する。
16. 条件 `'exclude_power_fit' in df.columns` を判定し、真なら内部処理を行う。
17. `df['exclude_power_fit']` に `df['exclude_power_fit'].astype(bool)` の結果を代入する。
18. 上の条件が偽の場合:
19. `df['exclude_power_fit']` に `False` の結果を代入する。
20. 条件 `'exclude_voltage_fit' in df.columns` を判定し、真なら内部処理を行う。
21. `df['exclude_voltage_fit']` に `df['exclude_voltage_fit'].astype(bool)` の結果を代入する。
22. 上の条件が偽の場合:
23. `df['exclude_voltage_fit']` に `False` の結果を代入する。
24. `clean` に `df.loc[~df['exclude_power_fit'] & ~df['exclude_voltage_fit']].copy()` の結果を代入する。
25. `daily` に `clean.groupby('local_date', dropna=False).agg(rows=('local_date', 'size'), max_s_km=('s_km', 'max'), power_rmse_w=('power_resid_w', rms), power_mae_w=('power_resid_w', lambda s: float(np.mean(np.abs(pd.to_numeric(s, errors='coerce'))))), voltage_rmse_v=('voltage_resid_v', rms), voltage_mae_v=('voltage_resid_v', lambda s: float(np.mean(np.abs(pd.to_numeric(s, errors='coerce'))))))).reset_index()` の結果を代入する。
26. `worst_power` に `clean.loc[clean['power_resid_w'].abs().nlargest(8).index, ['time_utc', 's_km', 'speed_kmh', 'power_resid_w', 'battery_power_w_obs', 'battery_power_w_pred']].copy()` の結果を代入する。
27. `worst_voltage` に `clean.loc[clean['voltage_resid_v'].abs().nlargest(8).index, ['time_utc', 's_km', 'speed_kmh', 'voltage_resid_v', 'battery_voltage_v_obs', 'battery_voltage_v_pred']].copy()` の結果を代入する。
28. `{'replay_csv': replay_csv, 'frame': df, 'clean_frame': clean, 'daily': daily, 'worst_power': worst_power, 'worst_voltage': worst_voltage}` を返す。

#### 代表コード断片

```
def load_replay_diagnostics(package_dir: Path, replay_csv: Path | None = None) -> Dict[str, object]:
    replay_csv = replay_csv or package_dir / "outputs" / "identification" / "replay_validation.csv"
    if not replay_csv.exists():
        return {
            "replay_csv": replay_csv,
            "frame": pd.DataFrame(),
            "clean_frame": pd.DataFrame(),
            "daily": pd.DataFrame(),
            "worst_power": pd.DataFrame(),
            "worst_voltage": pd.DataFrame(),
        }
```

```

df = pd.read_csv(replay_csv, low_memory=False)
fallback_ts = pd.to_datetime(
    df["time_utc"], format="mixed", utc=True, errors="coerce"
).dt.tz_convert("Australia/Darwin").dt.tz_localize(None)
if "time_local" in df.columns:
    ts = pd.to_datetime(df["time_local"], format="mixed", errors="coerce")
    if getattr(ts.dt, "tz", None) is not None:
        ts = ts.dt.tz_localize(None)
    ts = ts.fillna(fallback_ts)
else:
    ts = fallback_ts
df["local_date"] = ts.dt.strftime("%Y-%m-%d")
df["power_resid_w"] = pd.to_numeric(df["battery_power_w_obs"], errors="coerce") - pd.
to_numeric(df["battery_power_w_pred"], errors="coerce")
df["voltage_resid_v"] = pd.to_numeric(df["battery_voltage_v_obs"], errors="coerce") - pd.
to_numeric(df["battery_voltage_v_pred"], errors="coerce")
if "exclude_power_fit" in df.columns:
    df["exclude_power_fit"] = df["exclude_power_fit"].astype(bool)
else:
    df["exclude_power_fit"] = False
if "exclude_voltage_fit" in df.columns:
    df["exclude_voltage_fit"] = df["exclude_voltage_fit"].astype(bool)
else:
    df["exclude_voltage_fit"] = False

clean = df.loc[(~df["exclude_power_fit"]) & (~df["exclude_voltage_fit"])].copy()
...

```

## 7.9 L238 関数 locate\_fullsim\_manifest

- 行範囲: L238–L263
- 所属: module 直下
- 定義: `locate_fullsim_manifest(package_dir:Path,profile_yaml:Path)->Path|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `exists`, `get`, `is_absolute`, `read_text`, `resolve`, `safe_load`, `str`, `strip`
- 戻り値の要点: `None` / `configured if configured.exists()` else `None` / `path`
- 主なローカル代入名: `candidates`, `configured`, `path`, `profile_cfg`, `raw`, `sim`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. `profile_cfg` に `yaml.safe_load(profile_yaml.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
2. `sim` に `profile_cfg.get('simulation', {})` or `{}` の結果を代入する。
3. `raw` に `str(sim.get('latest_manifest_json', ''))` or `''` の結果を代入する。
4. 条件 `raw` を判定し、真なら内部処理を行う。
5. `configured` に `Path(raw)` の結果を代入する。
6. 条件 `not configured.is_absolute()` を判定し、真なら内部処理を行う。
7. `configured` に `ROOT / configured if configured.parts and configured.parts[0] == 'project_packages' else profile_yaml.parent / configured` の結果を代入する。
8. `configured` に `configured.resolve()` の結果を代入する。
9. `configured if configured.exists() else None` を返す。
10. `candidates` に `[package_dir / 'outputs' / 'prerace_fullsim_selflearned' / 'latest_simulation_run.json', package_dir / 'outputs' / 'prerace_final_selflearned' / 'latest_simulation_run.json', package_dir / 'outputs' / 'prerace' / 'latest_simulation_run.json']` の結果を代入する。
11. `candidates` を順に走査し、各要素を `path` に入れて処理する。
12. 条件 `path.exists()` を判定し、真なら内部処理を行う。
13. `path` を返す。
14. `None` を返す。

## 代表コード断片

```
def locate_fullsim_manifest(package_dir: Path, profile_yaml: Path) -> Path | None:
    profile_cfg = yaml.safe_load(profile_yaml.read_text(encoding="utf-8")) or {}
    sim = profile_cfg.get("simulation", {}) or {}
    raw = str(sim.get("latest_manifest_json", "")) or ""
    if raw:
        configured = Path(raw)
        if not configured.is_absolute():
            # Project profiles normally store workspace-relative output paths. Do
            # not fall back to an older run when this exact manifest is missing.
            configured = (
                ROOT / configured
                if configured.parts and configured.parts[0] == "project_packages"
                else profile_yaml.parent / configured
            )
        configured = configured.resolve()
        return configured if configured.exists() else None

    candidates = [
        package_dir / "outputs" / "prerace_fullsim_selflearned" / "latest_simulation_run.
json",
        package_dir / "outputs" / "prerace_final_selflearned" / "latest_simulation_run.json"
    ,
        package_dir / "outputs" / "prerace" / "latest_simulation_run.json",
    ]
    for path in candidates:
        if path.exists():
            return path
    return None
```

## 7.10 L266 関数 `resolve_manifest_artifact`

- 行範囲: L266–L274
- 所属: module 直下
- 定義: `resolve_manifest_artifact(manifest_path: Path, raw: str, package_dir: Path) -> Path`
- docstring: Resolve copied manifests whose original absolute path belongs to another host.
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_file`, `resolve_path`, `str`, `strip`
- 戻り値の要点: `resolve_path(package_dir, str(raw or "")) / path / local_sibling`
- 主なローカル代入名: `local_sibling`, `path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `path` に `Path(str(raw or "").strip())` の結果を代入する。
2. 条件 `path.is_file()` を判定し、真なら内部処理を行う。
3. `path` を返す。
4. `local_sibling` に `manifest_path.parent / path.name` の結果を代入する。
5. 条件 `local_sibling.is_file()` を判定し、真なら内部処理を行う。
6. `local_sibling` を返す。
7. `resolve_path(package_dir, str(raw or ""))` を返す。

### 代表コード断片

```
def resolve_manifest_artifact(manifest_path: Path, raw: str, package_dir: Path) -> Path:
    """Resolve copied manifests whose original absolute path belongs to another host."""
    path = Path(str(raw or "").strip())
    if path.is_file():
        return path
    local_sibling = manifest_path.parent / path.name
    if local_sibling.is_file():
        return local_sibling
    return resolve_path(package_dir, str(raw or ""))
```

## 7.11 L277 関数 `load_fullsim_summary`

- 行範囲: L277–L353
- 所属: module 直下
- 定義: `load_fullsim_summary(package_dir:Path,profile_yaml:Path,manifest_path:Path|None=None)->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `agg`, `clip`, `exists`, `fillna`, `get`, `getattr`, `groupby`, `issubset`, `items`, `loads`, `locate_fullsim_manifest`
- 戻り値の要点: `{'manifest_path': manifest_path, 'manifest': manifest, 'frame': df, 'daily': daily} / {'manifest_path': None, 'manifest': {}, 'frame': pd.DataFrame(), 'daily': pd.DataFrame()}`
- 主なローカル代入名: `aggregate`, `daily`, `daily_agg`, `df`, `dt_sec`, `energy_columns`, `local_ts`, `manifest`, `manifest_path`, `moving`, `name`, `optional_aggregates`, `out_csv`, `source`, `target`, `utc_ts`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 2、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

### 上から順の処理

1. `manifest_path` に `manifest_path` or `locate_fullsim_manifest(package_dir, profile_yaml)` の結果を代入する。
2. 条件 `manifest_path is None` を判定し、真なら内部処理を行う。
3. `{'manifest_path': None, 'manifest': {}, 'frame': pd.DataFrame(), 'daily': pd.DataFrame()}` を返す。
4. `manifest` に `json.loads(manifest_path.read_text(encoding='utf-8'))` の結果を代入する。
5. `out_csv` に `resolve_manifest_artifact(manifest_path, manifest.get('out_csv', ''), package_dir)` の結果を代入する。
6. `df` に `pd.DataFrame()` の結果を代入する。
7. `daily` に `pd.DataFrame()` の結果を代入する。
8. 条件 `out_csv.exists()` を判定し、真なら内部処理を行う。
9. `df` に `pd.read_csv(out_csv, low_memory=False)` の結果を代入する。
10. `utc_ts` に `pd.to_datetime(df['time_utc'], format='mixed', utc=True, errors='coerce').dt.tz_convert('Australia/Darwin').dt.` の結果を代入する。
11. 条件 `'time_local' in df.columns` を判定し、真なら内部処理を行う。
12. `local_ts` に `pd.to_datetime(df['time_local'], format='mixed', errors='coerce')` の結果を代入する。
13. 条件 `getattr(local_ts.dt, 'tz', None) is not None` を判定し、真なら内部処理を行う。

14. `local_ts` に `local_ts.dt.tz_localize(None)` の結果を代入する。
15. `local_ts` に `local_ts.fillna(utc_ts)` の結果を代入する。
16. 上の条件が偽の場合:
17. `local_ts` に `utc_ts` の結果を代入する。
18. `df['local_date']` に `local_ts.dt.strftime('%Y-%m-%d')` の結果を代入する。
19. `dt_sec` に `pd.to_numeric(df.get('step_dt_sec', 0.0), errors='coerce').fillna(0.0).clip(lower=0.0)` の結果を代入する。
20. `energy_columns` に `{'pv_energy_wh': 'P_pv', 'vehicle_load_energy_wh': 'P_vehicle_load_w', 'drive_dc_energy_wh': 'P_dc_to_drv', 'road_load_energy_wh': 'P_road_load', 'aux_energy_wh': 'P_aux', 'pack_net_energy_wh': 'P_pack', 'internal_loss_energy_wh': 'losses_int', 'line_loss_energy_wh': 'losses_line'}` の結果を代入する。
21. `energy_columns.items()` を順に走査し、各要素を `(target, source)` に入れて処理する。
22. 条件 `source in df.columns` を判定し、真なら内部処理を行う。
23. `df[target]` に `pd.to_numeric(df[source], errors='coerce').fillna(0.0) * dt_sec / 3600.0` の結果を代入する。
24. 条件 `{'eff_drv', 'v_exec_kmh'}.issubset(df.columns)` を判定し、真なら内部処理を行う。
25. `moving` に `pd.to_numeric(df['v_exec_kmh'], errors='coerce') > 1.0` の結果を代入する。
26. `df['eff_drv_moving']` に `pd.to_numeric(df['eff_drv'], errors='coerce').where(moving)` の結果を代入する。
27. `daily_agg` に `{'rows': ('local_date', 'size'), 'end_s_kmh': ('s_kmh', 'max'), 'min_soc': ('soc', 'min'), 'end_soc': ('soc_end' if 'soc_end' in df.columns else 'soc', 'last'), 'avg_v_exec_kmh': ('v_exec_kmh', 'mean'), 'avg_v_cmd_kmh': ('v_cmd_kmh', 'mean')}` を代入する。
28. `optional_aggregates` に `{'mean_poa_wm2': ('G_poa', 'mean'), 'max_poa_wm2': ('G_poa', 'max'), 'mean_ambient_c': ('Tamb_C', 'mean'), 'min_ambient_c': ('Tamb_C', 'min'), 'max_ambient_c': ('Tamb_C', 'max'), 'mean_cell_c': ('Tcell_C', 'mean'), 'mean_pv_w': ('P_pv', 'mean'), 'max_pv_w': ('P_pv', 'max'), 'max_discharge_current_a': ('I', 'max'), 'max_charge_current_a': ('I', 'min'), 'min_pack_voltage_v': ('V', 'min'), 'mean_drive_eff': ('eff_drv_moving', 'mean'), **{name: (name, 'sum') for name in energy_columns if name in df.columns}}` の結果を代入する。
29. `optional_aggregates.items()` を順に走査し、各要素を `(name, aggregate)` に入れて処理する。
30. 条件 `aggregate[0] in df.columns` を判定し、真なら内部処理を行う。
31. `daily_agg[name]` に `aggregate` の結果を代入する。
32. `daily` に `df.groupby('local_date', dropna=False).agg(**daily_agg).reset_index()` の結果を代入する。
33. `{'manifest_path': manifest_path, 'manifest': manifest, 'frame': df, 'daily': daily}` を返す。

#### 代表コード断片

```
def load_fullsim_summary(
    package_dir: Path,
    profile_yaml: Path,
    manifest_path: Path | None = None,
) -> Dict[str, object]:
    manifest_path = manifest_path or locate_fullsim_manifest(package_dir, profile_yaml)
    if manifest_path is None:
        return {
            "manifest_path": None,
            "manifest": {},
            "frame": pd.DataFrame(),
            "daily": pd.DataFrame(),
        }
```

```

manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
out_csv = resolve_manifest_artifact(manifest_path, manifest.get("out_csv", ""),
package_dir)
df = pd.DataFrame()
daily = pd.DataFrame()
if out_csv.exists():
    df = pd.read_csv(out_csv, low_memory=False)
    utc_ts = pd.to_datetime(
        df["time_utc"], format="mixed", utc=True, errors="coerce"
    ).dt.tz_convert("Australia/Darwin").dt.tz_localize(None)
    if "time_local" in df.columns:
        local_ts = pd.to_datetime(df["time_local"], format="mixed", errors="coerce")
        if getattr(local_ts.dt, "tz", None) is not None:
            local_ts = local_ts.dt.tz_localize(None)
        local_ts = local_ts.fillna(utc_ts)
    else:
        local_ts = utc_ts
    df["local_date"] = local_ts.dt.strftime("%Y-%m-%d")
    dt_sec = pd.to_numeric(df.get("step_dt_sec", 0.0), errors="coerce").fillna(0.0).clip(
lower=0.0)
    energy_columns = {
        "pv_energy_wh": "P_pv",
        "vehicle_load_energy_wh": "P_vehicle_load_w",
        "drive_dc_energy_wh": "P_dc_to_drv",
...

```

## 7.12 L401 関数 audit\_fullsim\_detail

- 行範囲: L401–L474
- 所属: module 直下
- 定義: `audit_fullsim_detail(package_dir:Path,manifest_path:Path|None,manifest:dict) ->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool, count_nonzero, difference, float, get, int, is_file, isclose, isfinite, len, list, max`
- 戻り値の要点: `{'available': True, 'detail_csv': str(detail_path), 'row_count': rows, 'column_count': len(header), 'required_column_count': len(DETAIL_REQUIRED_COLUMNS), 'missing_required_columns': missing, 'min_step_dt_sec': min_dt if np.isfinite(min_dt) else float('nan'), 'max_step_dt_sec': max_dt if np.isfinite(max_dt) else float('nan'), 'nominal_one_second_rows': nominal_rows, 'boundary_partial_rows': boundary_partial_rows, 'nonfinite_step_rows': nonfinite_step_rows, 'nonpositive_step_rows': nonpositive_rows, 'over_one_second_rows': over_one_second_rows, 'target_not_one_second_rows': target_not_one_second_rows, 'contract_pass': bool(not missing and rows > 0 and (nonfinite_step_rows == 0) and (nonpositive_rows == 0) and (over_one_second_rows == 0) and (target_not_one_second_rows == 0))} / {'available': False, 'reason': 'detail_csv_not_declared'} / {'available': False, 'reason': 'detail_csv_missing', 'detail_csv': str(detail_path)}`
- 主なローカル代入名: `boundary_partial_rows, chunk, column, detail_path, dt, finite, header, max_dt, min_dt, missing, nominal_rows, nonfinite_step_rows, nonpositive_rows, over_one_second_rows, raw, rows, target, target_not_one_second_rows, usecols`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 1、try 0

## この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

### 上から順の処理

1. `raw` に `str(manifest.get('detail_csv', '') or '').strip()` の結果を代入する。
2. 条件 `not raw or manifest_path is None` を判定し、真なら内部処理を行う。
3. `{'available': False, 'reason': 'detail_csv_not_declared'}` を返す。
4. `detail_path` に `resolve_manifest_artifact(manifest_path, raw, package_dir)` の結果を代入する。
5. 条件 `not detail_path.is_file()` を判定し、真なら内部処理を行う。
6. `{'available': False, 'reason': 'detail_csv_missing', 'detail_csv': str(detail_path)}` を返す。
7. `header` に `list(pd.read_csv(detail_path, nrows=0).columns)` の結果を代入する。
8. `missing` に `sorted(DETAIL_REQUIRED_COLUMNS.difference(header))` の結果を代入する。
9. `rows` に 0 の結果を代入する。
10. `nominal_rows` に 0 の結果を代入する。
11. `boundary_partial_rows` に 0 の結果を代入する。
12. `nonfinite_step_rows` に 0 の結果を代入する。
13. `nonpositive_rows` に 0 の結果を代入する。
14. `over_one_second_rows` に 0 の結果を代入する。
15. `target_not_one_second_rows` に 0 の結果を代入する。
16. `min_dt` に `float('inf')` の結果を代入する。
17. `max_dt` に `float('-inf')` の結果を代入する。
18. `usecols` に `[column for column in ('step_dt_sec', 'detail_target_dt_sec') if column in header]` の結果を代入する。
19. 条件 `'step_dt_sec' in usecols` を判定し、真なら内部処理を行う。
20. `pd.read_csv(detail_path, usecols=usecols, chunksize=200000)` を順に走査し、各要素を `chunk` に入れて処理する。
21. `dt` に `pd.to_numeric(chunk['step_dt_sec'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
22. `finite` に `dt[np.isfinite(dt)]` の結果を代入する。
23. `rows` を `Add` で更新する。
24. `nonfinite_step_rows` を `Add` で更新する。
25. 条件 `'detail_target_dt_sec' in chunk.columns` を判定し、真なら内部処理を行う。
26. `target` に `pd.to_numeric(chunk['detail_target_dt_sec'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
27. `target_not_one_second_rows` を `Add` で更新する。

28. 条件 `finite.size` を判定し、真なら内部処理を行う。
29. `min_dt` に `min(min_dt, float(np.min(finite)))` の結果を代入する。
30. `max_dt` に `max(max_dt, float(np.max(finite)))` の結果を代入する。
31. `nominal_rows` を `Add` で更新する。
32. `boundary_partial_rows` を `Add` で更新する。
33. `nonpositive_rows` を `Add` で更新する。
34. `over_one_second_rows` を `Add` で更新する。
35. `{'available': True, 'detail_csv': str(detail_path), 'row_count': rows, 'column_count': len(header), 'required_column_count': len(DETAIL_REQUIRED_COLUMNS), 'missing_required_columns': missing, 'min_step_dt_sec': min_dt if np.isfinite(min_dt) else float('nan'), 'max_step_dt_sec': max_dt if np.isfinite(max_dt) else float('nan'), 'nominal_one_second_rows': nominal_rows, 'boundary_partial_rows': boundary_partial_rows, 'nonfinite_step_rows': nonfinite_step_rows, 'nonpositive_step_rows': nonpositive_rows, 'over_one_second_rows': over_one_second_rows, 'target_not_one_second_rows': target_not_one_second_rows, 'contract_pass': bool(not missing and rows > 0 and (nonfinite_step_rows == 0) and (nonpositive_rows == 0) and (over_one_second_rows == 0) and (target_not_one_second_rows == 0))}` を返す。

#### 代表コード断片

```
def audit_fullsim_detail(
    package_dir: Path,
    manifest_path: Path | None,
    manifest: dict,
) -> Dict[str, object]:
    raw = str(manifest.get("detail_csv", "") or "").strip()
    if not raw or manifest_path is None:
        return {"available": False, "reason": "detail_csv_not_declared"}
    detail_path = resolve_manifest_artifact(manifest_path, raw, package_dir)
    if not detail_path.is_file():
        return {
            "available": False,
            "reason": "detail_csv_missing",
            "detail_csv": str(detail_path),
        }
    header = list(pd.read_csv(detail_path, nrows=0).columns)
    missing = sorted(DETAIL_REQUIRED_COLUMNS.difference(header))
    rows = 0
    nominal_rows = 0
    boundary_partial_rows = 0
    nonfinite_step_rows = 0
    nonpositive_rows = 0
    over_one_second_rows = 0
    target_not_one_second_rows = 0
    min_dt = float("inf")
    max_dt = float("-inf")
    usecols = [column for column in ("step_dt_sec", "detail_target_dt_sec") if column in header]
    if "step_dt_sec" in usecols:
        for chunk in pd.read_csv(detail_path, usecols=usecols, chunksize=200_000):
            dt = pd.to_numeric(chunk["step_dt_sec"], errors="coerce").to_numpy(dtype=float)
            finite = dt[np.isfinite(dt)]
            rows += int(len(chunk))
            nonfinite_step_rows += int(len(dt) - len(finite))
            if "detail_target_dt_sec" in chunk.columns:
                target = pd.to_numeric(
...

```

### 7.13 L477 関数 `interpolate_at_distance`

- 行範囲: L477–L493
- 所属: module 直下
- 定義: `interpolate_at_distance(df:pd.DataFrame,column:str,distance_km:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `dropna`, `float`, `groupby`, `interp`, `last`, `sort_values`, `to_numeric`, `to_numpy`
- 戻り値の要点: `float(np.interp(float(distance_km), x, y)) / float('nan') / float('nan') / float('nan')`
- 主なローカル代入名: `work`, `x`, `y`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

#### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

#### 上から順の処理

1. 条件 `df.empty or 's_km' not in df.columns or column not in df.columns` を判定し、真なら内部処理を行う。
2. `float('nan')` を返す。
3. `work` に `pd.DataFrame({'s_km': pd.to_numeric(df['s_km'], errors='coerce'), 'value': pd.to_numeric(df[column], errors='coerce')}).dropna()` の結果を代入する。
4. 条件 `work.empty` を判定し、真なら内部処理を行う。
5. `float('nan')` を返す。
6. `work` に `work.groupby('s_km', as_index=False)['value'].last().sort_values('s_km')` の結果を代入する。
7. `x` に `work['s_km'].to_numpy(dtype=float)` の結果を代入する。
8. `y` に `work['value'].to_numpy(dtype=float)` の結果を代入する。
9. 条件 `distance_km < x[0] or distance_km > x[-1]` を判定し、真なら内部処理を行う。
10. `float('nan')` を返す。
11. `float(np.interp(float(distance_km), x, y))` を返す。

#### 代表コード断片

```
def interpolate_at_distance(df: pd.DataFrame, column: str, distance_km: float) -> float:
    if df.empty or "s_km" not in df.columns or column not in df.columns:
        return float("nan")
    work = pd.DataFrame(
        {
            "s_km": pd.to_numeric(df["s_km"], errors="coerce"),
            "value": pd.to_numeric(df[column], errors="coerce"),
        }
    ).dropna()
    if work.empty:
        return float("nan")
    work = work.groupby("s_km", as_index=False)["value"].last().sort_values("s_km")
    x = work["s_km"].to_numpy(dtype=float)
    y = work["value"].to_numpy(dtype=float)
    if distance_km < x[0] or distance_km > x[-1]:
        return float("nan")
    return float(np.interp(float(distance_km), x, y))
```

### 7.14 L496 関数 moving\_speed\_in\_interval

- 行範囲: L496–L508
- 所属: module 直下
- 定義: `moving_speed_in_interval(df:pd.DataFrame,column:str,start_km:float,end_km:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dropna`, `float`, `mean`, `to_numeric`
- 戻り値の要点: `float(values.mean())` if not `values.empty` else `float('nan')` / `float('nan')`
- 主なローカル代入名: `distance`, `mask`, `speed`, `values`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

#### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

#### 上から順の処理

1. 条件 `df.empty or 's_km' not in df.columns or column not in df.columns` を判定し、真なら内部処理を行う。
2. `float('nan')` を返す。
3. `distance` に `pd.to_numeric(df['s_km'], errors='coerce')` の結果を代入する。
4. `speed` に `pd.to_numeric(df[column], errors='coerce')` の結果を代入する。
5. `mask` に `(distance > start_km) & (distance <= end_km) & (speed > 5.0)` の結果を代入する。
6. `values` に `speed.loc[mask].dropna()` の結果を代入する。
7. `float(values.mean())` if not `values.empty` else `float('nan')` を返す。

#### 代表コード断片

```
def moving_speed_in_interval(
    df: pd.DataFrame,
    column: str,
    start_km: float,
    end_km: float,
) -> float:
    if df.empty or "s_km" not in df.columns or column not in df.columns:
        return float("nan")
    distance = pd.to_numeric(df["s_km"], errors="coerce")
    speed = pd.to_numeric(df[column], errors="coerce")
    mask = (distance > start_km) & (distance <= end_km) & (speed > 5.0)
    values = speed.loc[mask].dropna()
    return float(values.mean()) if not values.empty else float("nan")
```

### 7.15 L511 関数 build\_human\_mpc\_distance\_comparison

- 行範囲: L511–L546
- 所属: module 直下
- 定義: `build_human_mpc_distance_comparison(replay_df:pd.DataFrame,fullsim_df:pd.DataFrame,retire_distance_km:float)->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `append`, `dropna`, `float`, `get`, `interpolate_at_distance`, `moving_speed_in_interval`, `sorted`, `to_numeric`
- 戻り値の要点: `pd.DataFrame(rows) / pd.DataFrame() / pd.DataFrame()`
- 主なローカル代入名: `end_km`, `endpoints`, `human_soc`, `human_soc_series`, `mpc_soc`, `mpc_soc_series`, `rows`, `start_km`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

#### この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

#### 上から順の処理

1. 条件 `replay_df.empty or fullsim_df.empty` を判定し、真なら内部処理を行う。
2. `pd.DataFrame()` を返す。
3. `endpoints` に `[500.0, 1000.0, 1500.0, 2000.0, 2500.0, float(retire_distance_km)]` の結果を代入する。
4. `endpoints` に `sorted({value for value in endpoints if 0.0 < value <= retire_distance_km})` の結果を代入する。
5. `human_soc_series` に `pd.to_numeric(replay_df.get('soc_pred'), errors='coerce').dropna()` の結果を代入する。
6. `mpc_soc_series` に `pd.to_numeric(fullsim_df.get('soc'), errors='coerce').dropna()` の結果を代入する。

7. 条件 `human_soc_series.empty` or `mpc_soc_series.empty` を判定し、真なら内部処理を行う。
8. `pd.DataFrame()` を返す。
9. `rows` に `[]` の結果を代入する。
10. `start_km` に `0.0` の結果を代入する。
11. `endpoints` を順に走査し、各要素を `end_km` に入れて処理する。
12. `human_soc` に `interpolate_at_distance(replay_df, 'soc_pred', end_km)` の結果を代入する。
13. `mpc_soc` に `interpolate_at_distance(fullsim_df, 'soc', end_km)` の結果を代入する。
14. `rows.append(...)` を実行する。
15. `start_km` に `end_km` の結果を代入する。
16. `pd.DataFrame(rows)` を返す。

#### 代表コード断片

```
def build_human_mpc_distance_comparison(
    replay_df: pd.DataFrame,
    fullsim_df: pd.DataFrame,
    retire_distance_km: float,
) -> pd.DataFrame:
    if replay_df.empty or fullsim_df.empty:
        return pd.DataFrame()
    endpoints = [500.0, 1000.0, 1500.0, 2000.0, 2500.0, float(retire_distance_km)]
    endpoints = sorted({value for value in endpoints if 0.0 < value <= retire_distance_km})
    human_soc_series = pd.to_numeric(replay_df.get("soc_pred"), errors="coerce").dropna()
    mpc_soc_series = pd.to_numeric(fullsim_df.get("soc"), errors="coerce").dropna()
    if human_soc_series.empty or mpc_soc_series.empty:
        return pd.DataFrame()

    rows = []
    start_km = 0.0
    for end_km in endpoints:
        human_soc = interpolate_at_distance(replay_df, "soc_pred", end_km)
        mpc_soc = interpolate_at_distance(fullsim_df, "soc", end_km)
        rows.append(
            {
                "segment_km": f"{start_km:.0f}-{end_km:.1f}",
                "end_s_km": end_km,
                "human_moving_speed_kmh": moving_speed_in_interval(
                    replay_df, "speed_kmh", start_km, end_km
                ),
                "mpc_moving_speed_kmh": moving_speed_in_interval(
                    fullsim_df, "v_exec_kmh", start_km, end_km
                ),
                "human_reconstructed_soc": human_soc,
                "mpc_no_trouble_soc": mpc_soc,
                "soc_gap_mpc_minus_human": mpc_soc - human_soc,
            }
        )
        start_km = end_km
    ...
```

## 7.16 L549 関数 `build_daily_progress_comparison`

- 行範囲: L549–L567

- 所属: module 直下
- 定義: `build_daily_progress_comparison(replay_df:pd.DataFrame, fullsim_df:pd.DataFrame)->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `groupby`, `max`, `merge`, `rename`, `reset_index`, `sort_values`
- 戻り値の要点: `out.reset_index(drop=True) / pd.DataFrame()`
- 主なローカル代入名: `human`, `mpc`, `out`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

## この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. 条件 `replay_df.empty or fullsim_df.empty or 'local_date' not in replay_df or ('local_date' not in fullsim_df)` を判定し、真なら内部処理を行う。
2. `pd.DataFrame()` を返す。
3. `human` に `replay_df.groupby('local_date', as_index=False)['s_km'].max().rename(columns={'s_km': 'human_end_s_km'})` の結果を代入する。
4. `mpc` に `fullsim_df.groupby('local_date', as_index=False)['s_km'].max().rename(columns={'s_km': 'mpc_end_s_km'})` の結果を代入する。
5. `out` に `human.merge(mpc, on='local_date', how='outer').sort_values('local_date')` の結果を代入する。
6. `out['mpc_progress_lead_km']` に `out['mpc_end_s_km'] - out['human_end_s_km']` の結果を代入する。
7. `out.reset_index(drop=True)` を返す。

## 代表コード断片

```
def build_daily_progress_comparison(
    replay_df: pd.DataFrame,
    fullsim_df: pd.DataFrame,
) -> pd.DataFrame:
    if replay_df.empty or fullsim_df.empty or "local_date" not in replay_df or "local_date"
    not in fullsim_df:
        return pd.DataFrame()
    human = (
        replay_df.groupby("local_date", as_index=False)["s_km"]
        .max()
        .rename(columns={"s_km": "human_end_s_km"})
    )
    mpc = (
        fullsim_df.groupby("local_date", as_index=False)["s_km"]
        .max()
        .rename(columns={"s_km": "mpc_end_s_km"})
    )
```

```

)
out = human.merge(mpc, on="local_date", how="outer").sort_values("local_date")
out["mpc_progress_lead_km"] = out["mpc_end_s_km"] - out["human_end_s_km"]
return out.reset_index(drop=True)

```

## 7.17 L570 関数 `write_human_mpc_distance_plot`

- 行範囲: L570–L624
- 所属: module 直下
- 定義: `write_human_mpc_distance_plot(replay_df:pd.DataFrame, fullsim_df:pd.DataFrame, comparison:pd.DataFrame, output_path:Path, retire_distance_km:float)->Path|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `apply`, `axvline`, `close`, `dropna`, `grid`, `groupby`, `last`, `legend`, `plot`, `savefig`, `set_xlabel`, `set_ylabel`
- 戻り値の要点: `output_path / None`
- 主なローカル代入名: `axes`, `color`, `df`, `fig`, `label`, `linestyle`, `value_col`, `work`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. 条件 `comparison.empty` を判定し、真なら内部処理を行う。
2. `None` を返す。
3. `(fig, axes)` に `plt.subplots(2, 1, figsize=(10.0, 7.0), sharex=True)` の結果を代入する。
4. `((replay_df, 'soc_pred', 'Historical replay reconstruction', 'black', '-'), (fullsim_df, 'soc', 'MPC no-trouble fullsim', '0.35', '-'))` を順に走査し、各要素を `(df, value_col, label, color, linestyle)` に入れて処理する。
5. `work` に `df[['s_km', value_col]].apply(pd.to_numeric, errors='coerce').dropna()` の結果を代入する。
6. `work` に `work.groupby('s_km', as_index=False)[value_col].last().sort_values('s_km')` の結果を代入する。
7. `axes[0].plot(...)` を実行する。
8. `axes[0].axvline(...)` を実行する。
9. `axes[0].set_ylabel(...)` を実行する。
10. `axes[0].grid(...)` を実行する。
11. `axes[0].legend(...)` を実行する。
12. `axes[1].plot(...)` を実行する。

13. axes[1].plot(...) を実行する。
14. axes[1].axvline(...) を実行する。
15. axes[1].set\_xlabel(...) を実行する。
16. axes[1].set\_ylabel(...) を実行する。
17. axes[1].grid(...) を実行する。
18. axes[1].legend(...) を実行する。
19. fig.suptitle(...) を実行する。
20. fig.tight\_layout(...) を実行する。
21. fig.savefig(...) を実行する。
22. plt.close(...) を実行する。
23. output\_path を返す。

#### 代表コード断片

```
def write_human_mpc_distance_plot(
    replay_df: pd.DataFrame,
    fullsim_df: pd.DataFrame,
    comparison: pd.DataFrame,
    output_path: Path,
    retire_distance_km: float,
) -> Path | None:
    if comparison.empty:
        return None
    fig, axes = plt.subplots(2, 1, figsize=(10.0, 7.0), sharex=True)
    for df, value_col, label, color, linestyle in (
        (replay_df, "soc_pred", "Historical replay reconstruction", "black", "-"),
        (fullsim_df, "soc", "MPC no-trouble fullsim", "0.35", "--"),
    ):
        work = df[["s_km", value_col]].apply(pd.to_numeric, errors="coerce").dropna()
        work = work.groupby("s_km", as_index=False)[value_col].last().sort_values("s_km")
        axes[0].plot(
            work["s_km"],
            work[value_col],
            label=label,
            color=color,
            linestyle=linestyle,
            linewidth=1.6,
        )
    axes[0].axvline(retire_distance_km, color="0.55", linestyle=":", linewidth=1.0)
    axes[0].set_ylabel("SoC [-]")
    axes[0].grid(alpha=0.25)
    axes[0].legend(loc="best")

    axes[1].plot(
        comparison["end_s_km"],
        comparison["human_moving_speed_kmh"],
        marker="o",
        label="Historical moving mean",
        color="black",
    )
    ...
```

## 7.18 L627 関数 `md_table`

- 行範囲: L627–L642
- 所属: module 直下
- 定義: `md_table(df:pd.DataFrame)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `copy`, `extend`, `is_float_dtype`, `itertuples`, `join`, `len`, `map`, `notna`, `str`
- 戻り値の要点: `'\n'.join(lines) / '_no data_'`
- 主なローカル代入名: `c`, `col`, `data`, `headers`, `lines`, `row`, `rows`, `sep`, `v`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

### この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

### 上から順の処理

1. 条件 `df.empty` を判定し、真なら内部処理を行う。
2. `'_no data_'` を返す。
3. `data` に `df.copy()` の結果を代入する。
4. `data.columns` を順に走査し、各要素を `col` に入れて処理する。
5. 条件 `pd.api.types.is_float_dtype(data[col])` を判定し、真なら内部処理を行う。
6. `data[col]` に `data[col].map(lambda v: f'{v:.4f}' if pd.notna(v) else '')` の結果を代入する。
7. `headers` に `[str(c) for c in data.columns]` の結果を代入する。
8. `rows` に `[[str(v) for v in row] for row in data.itertuples(index=False, name=None)]` の結果を代入する。
9. `sep` に `['—'] * len(headers)` の結果を代入する。
10. `lines` に `['|'+ '|'.join(headers)+'|', '|'+ '|'.join(sep)+'|']` の結果を代入する。
11. `lines.extend(...)` を実行する。
12. `'\n'.join(lines)` を返す。

#### 代表コード断片

```
def md_table(df: pd.DataFrame) -> str:
    if df.empty:
        return "_no data_"
    data = df.copy()
    for col in data.columns:
        if pd.api.types.is_float_dtype(data[col]):
            data[col] = data[col].map(lambda v: f"{v:.4f}" if pd.notna(v) else "")
    headers = [str(c) for c in data.columns]
    rows = [[str(v) for v in row] for row in data.itertuples(index=False, name=None)]
    sep = ["---"] * len(headers)
    lines = [
        "| " + " | ".join(headers) + " |",
        "| " + " | ".join(sep) + " |",
    ]
    lines.extend("| " + " | ".join(row) + " |" for row in rows)
    return "\n".join(lines)
```

### 7.19 L645 関数 tex\_kv\_table

- 行範囲: L645–L658
- 所属: module 直下
- 定義: `tex_kv_table(items:Iterable[tuple[str,object]])->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dedent`, `join`, `latex_escape`, `strip`
- 戻り値の要点: `textwrap.dedent(f'\n \\\begin{{longtable}}{{p{{0.46\\linewidth}}p{{0.36\\linewidth}}}\n \\\toprule\n 項目 & 値 \\\n \\\midrule\n \\\endhead\n {rows}\n \\\bottomrule\n \\\end{{longtable}}\n ').strip()`
- 主なローカル代入名: `k`, `rows`, `v`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

#### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

#### 上から順の処理

1. `rows` に `'\n'.join((f'{latex_escape(k)} & {latex_escape(v)} \\\n' for k, v in items))` の結果を代入する。
2. `textwrap.dedent(f'\n \\\begin{{longtable}}{{p{{0.46\\linewidth}}p{{0.36\\linewidth}}}\n \\\toprule\n 項目 & 値 \\\n \\\midrule\n \\\endhead\n {rows}\n \\\bottomrule\n \\\end{{longtable}}\n ').strip()` を返す。

#### 代表コード断片

```
def tex_kv_table(items: Iterable[tuple[str, object]]) -> str:
    rows = "\n".join(f"{latex_escape(k)} & {latex_escape(v)} \\\\" for k, v in items)
    return textwrap.dedent(
        f"""
        \\begin{{longtable}}{{p{{0.46\\linewidth}}p{{0.36\\linewidth}}}}
        \\toprule
        項目 & 値 \\\\"
        \\midrule
        \\endhead
        {rows}
        \\bottomrule
        \\end{{longtable}}
        """
    ).strip()
```

## 7.20 L661 関数 tex\_df\_table

- 行範囲: L661–L695
- 所属: module 直下
- 定義: `tex_df_table(df:pd.DataFrame,title:str)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `copy`, `dedent`, `is_float_dtype`, `isna`, `itertuples`, `join`, `latex_escape`, `len`, `map`, `max`, `ravel`
- 戻り値の要点: `textwrap.dedent(f'\n \\paragraph{{{latex_escape(title)}}}\n \\begin{{center}}\n {table}\n \\end{{center}}\n ').strip() / f'\n \\paragraph{{{latex_escape(title)}}} no data available.'`
- 主なローカル代入名: `c`, `col`, `colspec`, `data`, `headers`, `max_cell_len`, `row`, `rows`, `table`, `v`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 1、try 0

### この定義を読むための Python 構文

- `'<'>`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

### 上から順の処理

1. 条件 `df.empty` を判定し、真なら内部処理を行う。
2. `f'\n \\paragraph{{{latex_escape(title)}}} no data available.'` を返す。
3. `data` に `df.copy()` の結果を代入する。
4. `data.columns` を順に走査し、各要素を `col` に入れて処理する。
5. 条件 `pd.api.types.is_float_dtype(data[col])` を判定し、真なら内部処理を行う。

6. data[col] に data[col].map(lambda v: " if pd.isna(v) else f'{v:.4f} ') の結果を代入する。
7. headers に ' & '.join((latex\_escape(c) for c in data.columns)) + ' \\\\' の結果を代入する。
8. rows に '\n'.join((' & '.join((latex\_escape(v) for v in row)) + ' \\\\' for row in data.astype(str).itertuples(index=False, name=None))) の結果を代入する。
9. max\_cell\_len に max((len(str(value)) for value in data.to\_numpy().ravel()), default=0) の結果を代入する。
10. 条件 len(data.columns) == 2 and max\_cell\_len > 48 を判定し、真なら内部処理を行う。
11. colspec に 'p{0.25\\textwidth}p{0.68\\textwidth}' の結果を代入する。
12. 上の条件が偽の場合:
13. colspec に 'l' \* len(data.columns) の結果を代入する。
14. table に textwrap.dedent(f'\n \\begin{{tabular}}{{{colspec}}}\n \\toprule\n {headers}\n \\midrule\n {rows}\n \\bottomrule\n \\end{{tabular}}\n ').strip() の結果を代入する。
15. 条件 len(data.columns) >= 4 を判定し、真なら内部処理を行う。
16. table に f'\\resizebox{{{\\textwidth}}}{{!}}{{%n{table}\\n}}' の結果を代入する。
17. textwrap.dedent(f'\n \\paragraph{{{latex\_escape(title)}}}\n \\begin{{center}}\n {table}\n \\end{{center}}\n ').strip() を返す。

#### 代表コード断片

```
def tex_df_table(df: pd.DataFrame, title: str) -> str:
    if df.empty:
        return f"\\paragraph{{{latex_escape(title)}}} no data available."
    data = df.copy()
    for col in data.columns:
        if pd.api.types.is_float_dtype(data[col]):
            data[col] = data[col].map(lambda v: " if pd.isna(v) else f'{v:.4f}")
    headers = " & ".join(latex_escape(c) for c in data.columns) + r" \\"
    rows = "\n".join(" & ".join(latex_escape(v) for v in row) + r" \\" for row in data.
        astype(str).itertuples(index=False, name=None))
    max_cell_len = max((len(str(value)) for value in data.to_numpy().ravel()), default=0)
    if len(data.columns) == 2 and max_cell_len > 48:
        colspec = r"p{0.25\\textwidth}p{0.68\\textwidth}"
    else:
        colspec = "l" * len(data.columns)
    table = textwrap.dedent(
        f"""
        \\begin{{tabular}}{{{colspec}}}
        \\toprule
        {headers}
        \\midrule
        {rows}
        \\bottomrule
        \\end{{tabular}}
        """
    ).strip()
    if len(data.columns) >= 4:
        table = f'\\resizebox{{{\\textwidth}}}{{!}}{{%n{table}\\n}}'
    return textwrap.dedent(
        f"""
        \\paragraph{{{latex_escape(title)}}}
        \\begin{{center}}
        {table}
        \\end{{center}}
        """
    ).strip()
```

## 7.21 L698 関数 `build_report`

- 行範囲: L698–L1557
- 所属: module 直下
- 定義: `build_report(package_dir:Path,profile_yaml:Path,*,fit_summary_path:Path|None=None,replay_csv:Path|None=None,fullsim_manifest:Path|None=None)->tuple[Path,Path]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Path`, `Series`, `abs`, `any`, `append`, `audit_fullsim_detail`, `bool`, `build_daily_progress_comparison`, `build_human_mpc_distance_comparison`, `compile_tex`, `day_block_bootstrap_rmse`
- 戻り値の要点: (`md_path`, `tex_path.with_suffix('.pdf')`)
- 主なローカル代入名: `_`, `acceptance_yaml`, `active_maps`, `battery`, `battery_dynamic`, `boundary_flags`, `boundary_tolerance`, `boundary_warning`, `certificate_candidates`, `clean_replay`, `component`, `conditional_power_nrmse`, `counterfactual`, `counterfactual_analysis`, `counterfactual_path`, `counterfactual_scenario`, `cruise_70kmh`, `cruise_70kmh_rows`, `daily_progress_comparison`, `detail_audit`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 1、`try` 0

### この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

### 上から順の処理

1. `fit_summary_path` に `fit_summary_path` or `locate_fit_summary(package_dir, profile_yaml)` の結果を代入する。
2. `fit_summary` に `yaml.safe_load(fit_summary_path.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
3. `profile` に `yaml.safe_load(profile_yaml.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
4. 条件 `replay_csv is None` を判定し、真なら内部処理を行う。
5. `versioned_replay` に `fit_summary_path.parent / 'replay_validation.csv'` の結果を代入する。
6. 条件 `versioned_replay.exists()` を判定し、真なら内部処理を行う。
7. `replay_csv` に `versioned_replay` の結果を代入する。
8. `replay` に `load_replay_diagnostics(package_dir, replay_csv)` の結果を代入する。
9. `end_to_end_replay_path` に `fit_summary_path.parent / 'replay_validation_end_to_end.csv'` の結果を代入する。

10. `end_to_end_replay` に `pd.read_csv(end_to_end_replay_path, low_memory=False)` if `end_to_end_replay_path.is_file()` else `replay.get('frame', pd.DataFrame())` の結果を代入する。
11. `(weather_daily, _)` に `weather_and_cruise_metrics(end_to_end_replay)` の結果を代入する。
12. `(_, cruise_70kmh)` に `weather_and_cruise_metrics(replay.get('frame', pd.DataFrame()))` の結果を代入する。
13. `cruise_70kmh_rows` に `pd.DataFrame([{'metric': key, 'value': value} for key, value in cruise_70kmh.items()])` の結果を代入する。
14. `fullsim` に `load_fullsim_summary(package_dir, profile_yaml, fullsim_manifest)` の結果を代入する。
15. `detail_audit` に `audit_fullsim_detail(package_dir, fullsim.get('manifest_path'), fullsim.get('manifest', {}))` or `{}` の結果を代入する。
16. `detail_audit_rows` に `pd.DataFrame([{'metric': key, 'value': ', '.join(value) if isinstance(value, list) else value} for key, value in detail_audit.items() if key != 'detail_csv'])` の結果を代入する。
17. `versioned_terminal_consistency` に `fit_summary_path.parent / 'terminal_soc_consistency.yaml'` の結果を代入する。
18. `terminal_consistency_path` に `versioned_terminal_consistency` if `versioned_terminal_consistency.exists()` else `package_dir / 'outputs' / 'identification' / 'terminal_soc_consistency.yaml'` の結果を代入する。
19. `terminal_consistency` に `yaml.safe_load(terminal_consistency_path.read_text(encoding='utf-8'))` or `{}` if `terminal_consistency_path.exists()` else `{}` の結果を代入する。
20. `report_dir` に `package_dir / 'outputs' / 'reports'` の結果を代入する。
21. `ensure_dir(...)` を実行する。
22. `stem` に `f'{profile_yaml.stem}_fit_fullsim_report'` の結果を代入する。
23. `md_path` に `report_dir / f'{stem}.md'` の結果を代入する。
24. `tex_path` に `report_dir / f'{stem}.tex'` の結果を代入する。
25. `versioned_current_maps` に `fit_summary_path.parent / 'current_maps_and_coefficients.md'` の結果を代入する。
26. `release_current_maps` に `report_dir / 'current_maps_and_coefficients.md'` の結果を代入する。
27. 条件 `versioned_current_maps.exists()` を判定し、真なら内部処理を行う。
28. `release_current_maps.write_bytes(...)` を実行する。
29. `battery` に `fit_summary.get('battery_fit', {})` or `{}` の結果を代入する。
30. `battery_dynamic` に `fit_summary.get('battery_dynamic_fit', {})` or `{}` の結果を代入する。
31. `motion` に `fit_summary.get('motion_fit', {})` or `{}` の結果を代入する。
32. `pv` に `fit_summary.get('pv_fit', {})` or `{}` の結果を代入する。
33. `metrics` に `fit_summary.get('validation_metrics', {})` or `{}` の結果を代入する。
34. `solar_calibration` に `(fit_summary.get('fit_plan', {}))` or `{}` の結果を代入し、`.get('solar_measurement_calibration', {})` or `{}` の結果を代入する。
35. `model_cfg` に `profile.get('model', {})` or `{}` の結果を代入する。
36. `clean_replay` に `replay.get('clean_frame', pd.DataFrame())` の結果を代入する。
37. `power_bootstrap` に `day_block_bootstrap_rmse(clean_replay, 'power_resid_w', seed=20260714)` の結果を代入する。

38. `voltage_bootstrap` に `day_block_bootstrap_rmse(clean_replay, 'voltage_resid_v', seed=20260715)` の結果を代入する。
39. `observed_power` に `pd.to_numeric(clean_replay.get('battery_power_w_obs', pd.Series(dtype=float)), errors='coerce').dropna()` の結果を代入する。
40. `observed_power_rms` に `float(np.sqrt(np.mean(observed_power.to_numpy(dtype=float) ** 2))) if not observed_power.empty else float('nan')` の結果を代入する。
41. `conditional_power_nrmse` に `float(power_bootstrap['rmse'])/observed_power_rms if np.isfinite(observed_power_rms) and observed_power_rms > 0.0 else float('nan')` の結果を代入する。
42. `fitted_e_nom_wh` に `float(battery.get('e_nom_wh', float('nan')))` の結果を代入する。
43. `fitted_rint_scale` に `float(battery.get('rint_scale', float('nan')))` の結果を代入する。
44. `fitted_eta_charge` に `float(battery.get('eta_charge', float('nan')))` の結果を代入する。
45. `fitted_q_nom_ah` に `float(model_cfg.get('Q_nom_Ah', float('nan')))` の結果を代入する。
46. `physical_parameter_gate_pass` に `bool(np.isfinite(fitted_e_nom_wh) and BATTERY_E_NOM_MIN_WH <= fitted_e_nom_wh <= BATTERY_E_NOM_MAX_WH and np.isfinite(fitted_rint_scale) and (BATTERY_RINT_SCALE_MIN <= fitted_rint_scale <= BATTERY_RINT_SCALE_MAX) and np.isfinite(fitted_eta_charge) and (BATTERY_ETA_CHARGE_MIN <= fitted_eta_charge <= BATTERY_ETA_CHARGE_MAX) and np.isfinite(fitted_q_nom_ah) and (fitted_q_nom_ah > 0.0))` の結果を代入する。
47. `boundary_tolerance` に `0.0001` の結果を代入する。
48. `boundary_flags` に `{'e_nom_at_min': abs(fitted_e_nom_wh - BATTERY_E_NOM_MIN_WH) <= boundary_tolerance, 'e_nom_at_max': abs(fitted_e_nom_wh - BATTERY_E_NOM_MAX_WH) <= boundary_tolerance, 'rint_scale_at_min': abs(fitted_rint_scale - BATTERY_RINT_SCALE_MIN) <= boundary_tolerance, 'rint_scale_at_max': abs(fitted_rint_scale - BATTERY_RINT_SCALE_MAX) <= boundary_tolerance, 'eta_charge_at_min': abs(fitted_eta_charge - BATTERY_ETA_CHARGE_MIN) <= boundary_tolerance, 'eta_charge_at_max': abs(fitted_eta_charge - BATTERY_ETA_CHARGE_MAX) <= boundary_tolerance}` の結果を代入する。
49. `boundary_warning` に `any(boundary_flags.values())` の結果を代入する。
50. `residual_precision_gate_pass` に `bool(np.isfinite(conditional_power_nrmse) and conditional_power_nrmse <= 0.15 and np.isfinite(float(voltage_bootstrap['rmse'])) and (float(voltage_bootstrap['rmse']) <= 1.0))` の結果を代入する。
51. `terminal` に `fit_summary.get('terminal_anchor', {})` or `{}` の結果を代入する。
52. `active_maps` に `fit_summary.get('active_maps', {})` or `{}` の結果を代入する。
53. `map_shape_fit` に `fit_summary.get('map_shape_fit', {})` or `{}` の結果を代入する。
54. `map_rows` に `pd.DataFrame([{'map': name, 'path': path} for name, path in sorted(active_maps.items())])` の結果を代入する。
55. `shape_rows` に `pd.DataFrame([{'map': name, 'samples': values.get('sample_count', ''), 'reason': values.get('reason', ''), 'rmse_before': values.get('rmse_before', ''), 'rmse_after': values.get('rmse_after', ''), 'corr_min': values.get('correction_min', ''), 'corr_max': values.get('correction_max', '')} for name, values in sorted(map_shape_fit.items()) if isinstance(values, dict) and 'name' in values])` の結果を代入する。
56. `evidence_bundle` に `fit_summary.get('evidence_bundle', {})` or `{}` の結果を代入する。
57. `grounded_summary_path` に `resolve_path(package_dir, str(evidence_bundle.get('grounded_map_summary_yaml', '')) or '')` の結果を代入する。
58. `grounded_summary` に `yaml.safe_load(grounded_summary_path.read_text(encoding='utf-8'))` or `{}` if `grounded_summary_path.is_file()` else `{}` の結果を代入する。

59. `grounded_rows_data` に [] の結果を代入する。
60. `grounded_summary.items()` を順に走査し、各要素を (`component`, `values`) に入れて処理する。
61. 条件 `not isinstance(values, dict)` を判定し、真なら内部処理を行う。
62. `Continue` 文を実行する。
63. `sources` に `[Path(str(value)).name for key, value in values.items() if str(key).startswith('source_') and isinstance(value, str)]` の結果を代入する。
64. `references` に `[f'{key}={value}' for key, value in values.items() if isinstance(value, (int, float)) and (not isinstance(value, bool))]` の結果を代入する。
65. `grounded_rows_data.append(...)` を実行する。
66. `grounded_rows` に `pd.DataFrame(grounded_rows_data)` の結果を代入する。
67. `grounded_detail_tex` に `'\n'.join(('\\subsubsection*{' + latex_escape(row['component']) + '}' + '\n' + tex_kv_table([('source_files', row['source_files']), ('physical/test basis', row['method']), ('reference values', row['reference_values'])]) for row in grounded_rows_data))` の結果を代入する。
68. `sim_cfg` に `profile.get('simulation', {})` or `{}` の結果を代入する。
69. `manifest` に `fullsim.get('manifest', {})` or `{}` の結果を代入する。
70. `solver_rows` に `manifest.get('upper_solver_diagnostics', [])` or `[]` の結果を代入する。
71. `certificate_candidates` に `int(sum((int(row.get('discrete_grid_candidates', 0)) or 0) for row in solver_rows if isinstance(row, dict)))` の結果を代入する。
72. `finite_library_candidates` に `int(sum((int(row.get('finite_library_candidates', 0)) or 0) for row in solver_rows if isinstance(row, dict)))` の結果を代入する。
73. `selected_controls` に `[row.get('selected_x', []) for row in solver_rows if isinstance(row, dict)]` の結果を代入する。
74. `retire_distance_km` に `float(terminal.get('s_km', 2831.0))` or `2831.0` の結果を代入する。
75. `distance_comparison` に `build_human_mpc_distance_comparison(replay.get('frame', pd.DataFrame()), fullsim.get('frame', pd.DataFrame()), retire_distance_km)` の結果を代入する。
76. `distance_comparison_display` に `distance_comparison.rename(columns={'segment_km': 'segment', 'human_moving_speed_kmh': 'v_human_kmh', 'mpc_moving_speed_kmh': 'v_mpc_kmh', 'human_reconstructed_soc': 'z_human_segment', 'mpc_no_trouble_soc': 'z_mpc', 'soc_gap_mpc_minus_human': 'z_gap'})` の結果を代入する。
77. 条件 `not distance_comparison_display.empty` を判定し、真なら内部処理を行う。
78. `distance_comparison_display` に `distance_comparison_display[['segment', 'v_human_kmh', 'v_mpc_kmh', 'z_human_segment', 'z_mpc', 'z_gap']]` の結果を代入する。
79. `daily_progress_comparison` に `build_daily_progress_comparison(replay.get('frame', pd.DataFrame()), fullsim.get('frame', pd.DataFrame()))` の結果を代入する。
80. `distance_plot_path` に `write_human_mpc_distance_plot(replay.get('frame', pd.DataFrame()), fullsim.get('frame', pd.DataFrame()), distance_comparison, report_dir / f'{stem}_human_mpc_distance.jpg', retire_distance_km)` の結果を代入する。

#### 代表コード断片

```
def build_report(
    package_dir: Path,
    profile_yaml: Path,
    *,
    fit_summary_path: Path | None = None,
    replay_csv: Path | None = None,
    fullsim_manifest: Path | None = None,
) -> tuple[Path, Path]:
    fit_summary_path = fit_summary_path or locate_fit_summary(package_dir, profile_yaml)
    fit_summary = yaml.safe_load(fit_summary_path.read_text(encoding="utf-8")) or {}
    profile = yaml.safe_load(profile_yaml.read_text(encoding="utf-8")) or {}
    if replay_csv is None:
        versioned_replay = fit_summary_path.parent / "replay_validation.csv"
        if versioned_replay.exists():
            replay_csv = versioned_replay
    replay = load_replay_diagnostics(package_dir, replay_csv)
    end_to_end_replay_path = fit_summary_path.parent / "replay_validation_end_to_end.csv"
    end_to_end_replay = (
        pd.read_csv(end_to_end_replay_path, low_memory=False)
        if end_to_end_replay_path.is_file()
        else replay.get("frame", pd.DataFrame())
    )
    weather_daily, _ = weather_and_cruise_metrics(end_to_end_replay)
    _, cruise_70kmh = weather_and_cruise_metrics(
        replay.get("frame", pd.DataFrame())
    )
    cruise_70kmh_rows = pd.DataFrame(
        [{"metric": key, "value": value} for key, value in cruise_70kmh.items()]
    )
    fullsim = load_fullsim_summary(package_dir, profile_yaml, fullsim_manifest)
    detail_audit = audit_fullsim_detail(
        package_dir,
        fullsim.get("manifest_path"),
        fullsim.get("manifest", {}) or {},
    )
    ...
```

## 7.22 L1560 関数 main

- 行範囲: L1560–L1601
- 所属: module 直下
- 定義: `main()->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `add_argument`, `build_report`, `dumps`, `locate_package_dir`, `parse_args`, `print`, `resolve_path`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `ap`, `args`, `fit_summary_path`, `fullsim_manifest`, `md_path`, `package_dir`, `pdf_path`, `profile_yaml`, `replay_csv`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

## この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. ap に argparse.ArgumentParser() の結果を代入する。
2. ap.add\_argument(...) を実行する。
3. ap.add\_argument(...) を実行する。
4. ap.add\_argument(...) を実行する。
5. ap.add\_argument(...) を実行する。
6. args に ap.parse\_args() の結果を代入する。
7. profile\_yaml に resolve\_path(ROOT, args.profile) の結果を代入する。
8. package\_dir に locate\_package\_dir(profile\_yaml) の結果を代入する。
9. fit\_summary\_path に resolve\_path(ROOT, args.fit\_summary) if args.fit\_summary else None の結果を代入する。
10. replay\_csv に resolve\_path(ROOT, args.replay\_csv) if args.replay\_csv else None の結果を代入する。
11. fullsim\_manifest に resolve\_path(ROOT, args.fullsim\_manifest) if args.fullsim\_manifest else None の結果を代入する。
12. (md\_path, pdf\_path) に build\_report(package\_dir, profile\_yaml, fit\_summary\_path=fit\_summary\_path, replay\_csv=replay\_csv, fullsim\_manifest=fullsim\_manifest) の結果を代入する。
13. print(...) を実行する。

### 代表コード断片

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--profile", required=True)
    ap.add_argument(
        "--fit-summary",
        default="",
        help="Optional fit-summary YAML. Defaults to the package generic summary.",
    )
    ap.add_argument(
        "--replay-csv",
        default="",
        help="Optional replay-validation CSV. Defaults to replay_validation.csv.",
    )
    ap.add_argument(
        "--fullsim-manifest",
        default="",
        help="Optional exact full-simulation manifest, including a manifest copied from another host.",
    )
    args = ap.parse_args()
    profile_yaml = resolve_path(ROOT, args.profile)
    package_dir = locate_package_dir(profile_yaml)
    fit_summary_path = resolve_path(ROOT, args.fit_summary) if args.fit_summary else None
    replay_csv = resolve_path(ROOT, args.replay_csv) if args.replay_csv else None
    fullsim_manifest = resolve_path(ROOT, args.fullsim_manifest) if args.fullsim_manifest else None
```

```

md_path, pdf_path = build_report(
    package_dir,
    profile_yaml,
    fit_summary_path=fit_summary_path,
    replay_csv=replay_csv,
    fullsim_manifest=fullsim_manifest,
)
print(
    json.dumps(
        {
            "profile_yaml": str(profile_yaml),
        }
    )
)
...

```

## 7.23 CLI 引数

| 行    | 引数                 |
|------|--------------------|
| 1562 | --profile          |
| 1563 | --fit-summary      |
| 1568 | --replay-csv       |
| 1573 | --fullsim-manifest |

## 8 処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

## 9 このファイルの位置づけ

この資料群では、scripts/generate\_fit\_fullsim\_report.py を **report** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。